

Heurística Manual: Análisis de Transiciones

Síntesis de Directores para Instancias LTS

Dario Turco

Quiero resolver todas las instancias
de benchmark.

OBJETIVO



DERROTAR CM-15-15

imgflip.com

CM es muy difícil, así que vamos a resolver TL con una heurística hecha a mano.

TL – Director para TL-2-2

Instance: TL-2-2

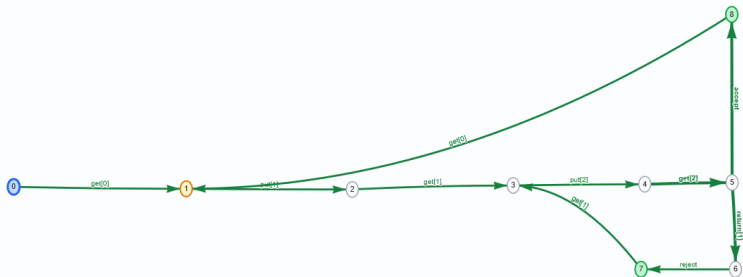
Mode: GR1

Heuristic: HAND

Steps: 10

Director: 10

File: python\results\traces\TL\HAND\TL-2-2.sol



⏮ Start

◀ Back

▶ Play

Fwd ▶

End ⏭

Step 10 / 10

Hide Unknown

Show Names

Hide Director

🔍

Exploration complete — REALIZABLE ✓

Director transitions:

[1] Working[1]|Operative[0]|Working[0]|Operative[0]|Idle|off#0 —[put[1]]→ Working[0]|Operative[1]|Working[0]|Operative[0]|Idle|off#0

[2] Working[0]|Operative[1]|Working[0]|Operative[0]|Idle|on#0 —[get[1]]→ Working[0]|Operative[0]|Working[1]|Operative[0]|Idle|off#0

Descomposición estructural:

- Cadena $get[0] \rightarrow put[1] \rightarrow get[1] \rightarrow \dots \rightarrow put[n] \rightarrow get[n]$: $2n+1$ transiciones
- Desde $get[n]$: 5 expansiones para cerrar ciclos
 - $return[1]$
 - $reject \rightarrow$ estado ganador $\rightarrow get$ (retorno)
 - $accept \rightarrow$ estado marcado $\rightarrow get$ (retorno)

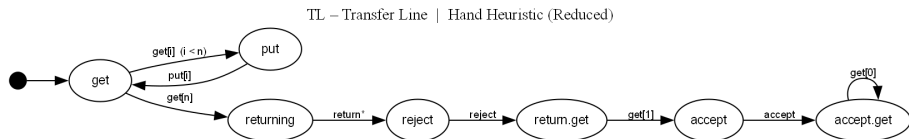
n	1	3	5	7	10	15
$T(n)$	8	12	16	20	26	36

Esto nos dice que el director lo podemos encontrar siempre expandiendo $T(n)$ transiciones:

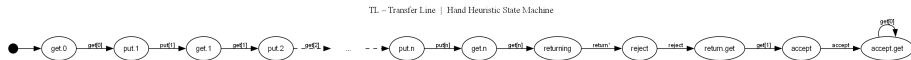
$$T(n) = 1 + 2n + 2 + 3 = 2n + 6$$

Podemos armar una máquina de estados para expandir estas transiciones:

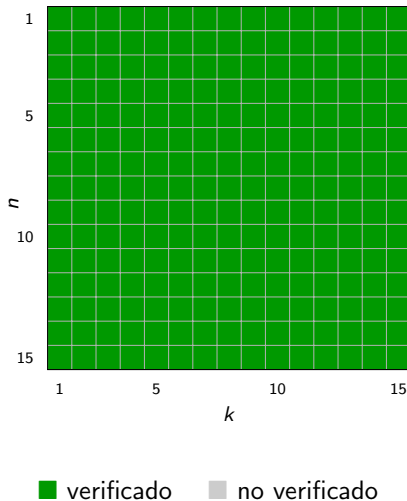
TL – Heurística (reducida)



TL – Heurística (extendida)

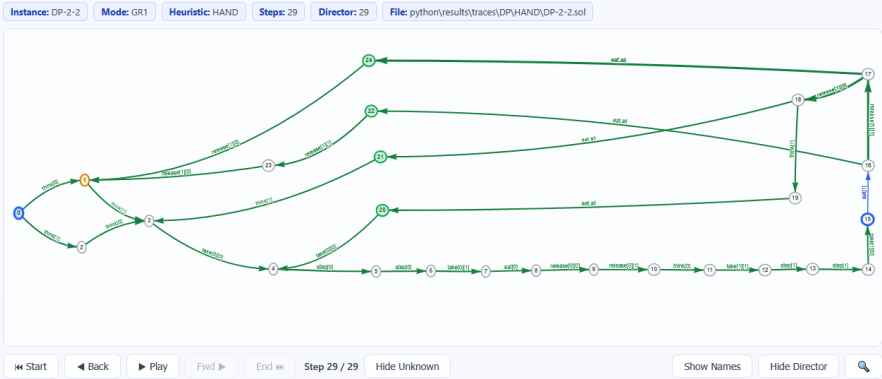


TL – Instancias verificadas



Ahora vamos a DP

DP – Director para DP-2-2



DP – Dining Philosophers

Fases:

- 1 **Órdenes de pensar** — todos los posibles órdenes en que piensan los n filósofos: $n \cdot 2^{n-1}$
- 2 **Alimentar** — alimentar a los n filósofos (crece con k): $n(k + 5) + n$
- 3 **Retorno** — volver desde estado marcado (constante): 9

Fórmula:

$$T_{\text{DP}}(n, k) = \underbrace{n \cdot 2^{n-1}}_{\text{órdenes de pensar}} + \underbrace{n(k + 5) + n}_{\text{alimentar}} + \underbrace{9}_{\text{retorno}}$$

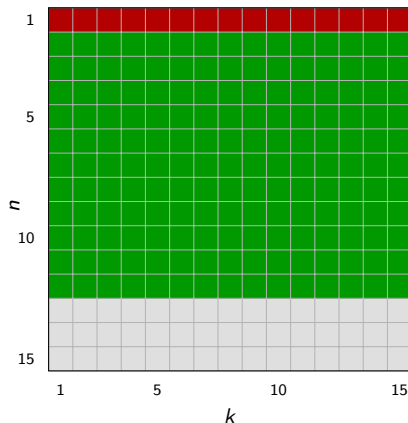
DP – Valores de ejemplo

$n \setminus k$	1	3	5	10	15
2	27	31	35	45	55
3	42	48	54	69	84
5	124	134	144	169	194
10	5199	5219	5239	5289	5339
11	11350	11372	11394	11449	11504
12	24669	24693	24717	24777	24837

Todos los valores verificados empíricamente contra el director sintetizado.

$$T_{DP}(2, 2) = 29 \qquad T_{DP}(15, 15) = 246084$$

DP – Instancias verificadas



■ verificado ■ no resoluble ($n=1$) ■ demasiado grande para resolver

DP – Director (extendido)



DP – Director (reducido)

Imagen pendiente — heuristic_DP_reduced.png

Cat and Mouse tiene **dos fases**:

A — *Aproximación* El ratón se mueve hacia el objetivo. Transiciones hasta llegar al goal.

B — *Retorno* El ratón está en el objetivo; el gato recorre sus posiciones hasta volver al inicio del ciclo.

$$T_{\text{CM}}(n, k) = A(n, k) + B(n, k)$$

CM – Cat and Mouse ($n = 1$, simplificado)

Para $n = 1$ la fórmula se simplifica a:

$$T_{\text{CM}}(1, k) = 3k^2 + 12k + 6$$

Dos fases estructurales:

Aproximación $A(1, k)$:

$$A = 3k^2 - 2k + 1$$

El ratón toma k pasos hacia el centro;
el gato lo sigue desde el borde.

Retorno $B(1, k)$:

$$B = 14k + 5$$

El ratón permanece en el centro; el
gato recorre las $2k+1$ posiciones.

$$A + B = (3k^2 - 2k + 1) + (14k + 5) = 3k^2 + 12k + 6 \quad \checkmark$$

CM – Fórmula general

Retorno:

$$B(n, k) = (2k+1)^{n-1} \cdot [(n+3)(2k+1) + n \cdot 2^{n-1}(6k+1)]$$

Aproximación: con $f(n, i) = (n+1)i^n + n(3i-1)(2i+1)^{n-1} + (i+1)^n$

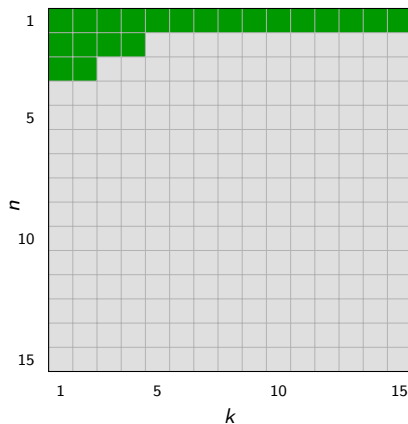
$$A(n, k) = 1 + \sum_{i=1}^{k-1} f(n, i) + n \cdot k^n, \quad T_{\text{CM}} = A + B$$

$n \backslash k$	1	2	3
1	21	42	69
2	132	413	886
3	922	4741	?

? = demasiado grande para verificar empíricamente

RA expande exactamente la misma cantidad ✓

CM – Instancias verificadas



■ verificado ■ demasiado grande para resolver

$$T_{\text{CM}}(15, 15) = 16\,929\,109\,944\,024\,661\,913\,431\,311\,618 \approx 10^{28,23}$$

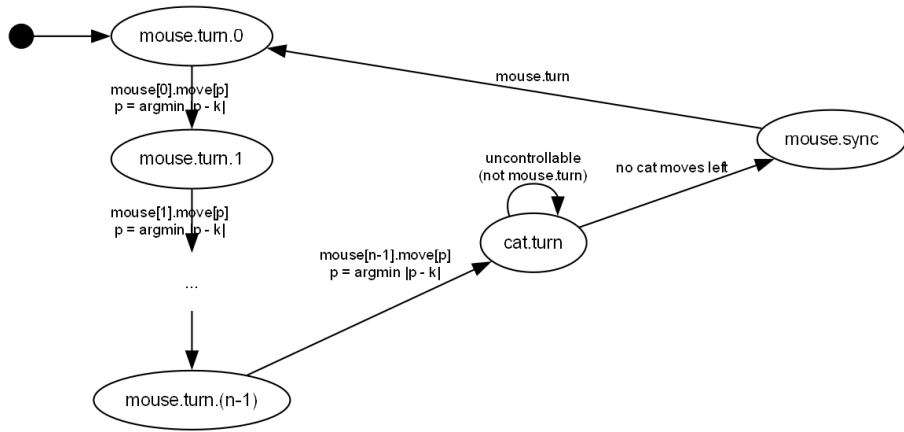
CM – ¿Cuánto tardaría resolverlo?

Expandiendo 1 transición/ns: $\approx$ **536 mil millones de años**
($\sim 39\times$ la edad del universo)

Imagen pendiente — meme esqueleto

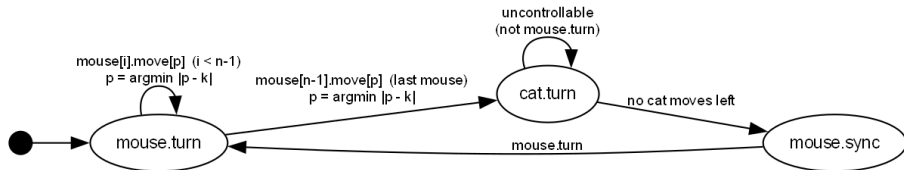
CM – Director (no compacto)

CM – Cat and Mouse | Hand Heuristic State Machine



CM – Director (compacto)

CM – Cat and Mouse | Hand Heuristic (Reduced)



Preguntas

❶ **¿La heurística manual es el director?**

Reconoce solo transiciones del director, pero ¿eso garantiza que es el director?

❷ **¿Se puede encontrar la heurística de AT conociendo solo AT-2-k?**

¿Alcanza con instancias pequeñas para derivar la heurística general?

❸ **¿Se pueden aprender estas heurísticas automáticamente?**

Conociendo el director de la instancia 2-2, ¿puede una máquina inferir la heurística general?

❹ **¿Una heurística general implica un conjunto suficiente de features?**

Si conocemos la heurística para toda una familia, ¿podemos identificar los features que resuelven todas sus instancias?

Estructura especial: solo estudiado para $n = k$

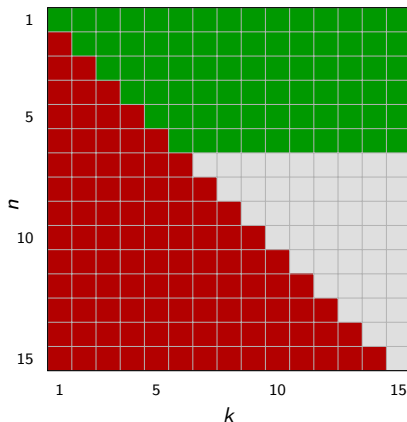
Fórmula: aún no derivada — la complejidad estructural impide una forma cerrada simple.

RA no es óptimo para AT

RA expande *más* transiciones que las del director.

La Heurística Manual supera estrictamente a RA en esta familia de instancias.

AT – Instancias verificadas



■ verificado ■ no resoluble ($k < n$) ■ demasiado grande para resolver

Todas las instancias verificadas logran **100 % de eficiencia**.

¿Se puede resolver AT-3-3 aprendiendo de AT-2-2?

Idea

Los directores de instancias pequeñas revelan la estructura del director general.

- AT-1-1, AT-2-2 son resolubles y sus directores son conocidos
- AT-3-3 es demasiado grande para resolver directamente
- **Pregunta:** ¿es posible *derivar* el director de AT-3-3 a partir del patrón observado en AT-1-1 y AT-2-2?

Las siguientes slides muestran los directores de AT-1-1, AT-2-2 y AT-3-3 para buscar el patrón.

Imagen pendiente — AT-1-1

Imagen pendiente — AT-2-2